

Experiment-8

Support Vector Machines

Aim:

To implement Support Vector Machine classifiers using linear and non-linear kernels for classification tasks, and to evaluate their performance on linearly and non-linearly separable datasets.

Theory:

Support Vector Machines (SVMs) were formally established as a statistical learning framework grounded in the principle of Structural Risk Minimization (SRM), by Cortes and Vapnik (1995). The fundamental objective of SVM resides in determining an optimal decision boundary that maximizes the separation margin between different classes in a given feature space.

The goal of the Support Vector Machine is to find the hyperplane that maximizes the margin between two classes. This principle ensures improved generalization performance by minimizing the upper bound on the generalization error rather than merely minimizing empirical risk.

I. Hyperplane: -

A hyperplane is a **flat affine subspace of one dimension less than its ambient space**. In an n -dimensional space, a hyperplane has $(n-1)$ dimensions. For a linearly separable dataset, the SVM seeks a separating hyperplane defined by:

$$w^T x + b = 0$$

where w is the weight vector which is normal to the hyperplane and b is the bias term.

Note - An optimal hyperplane is the one that maximizes the margin between the classes.

II. Support Vectors: -

A margin is defined as the perpendicular distance between the hyperplane and the nearest data points from each class. These points are known as support vectors.

III. Margin Maximization: -

The margin maximization problem can be formulated as:

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

with constraint,

$$y_i(w \cdot x_i + b) \geq 1$$

for all training samples (x_i, y_i) . This formulation ensures that the separating hyperplane lies as far as possible from the closest data points, thereby enhancing the classifier's ability to generalize to unseen data.

- IV. **Kernels:** - SVM employs the kernel trick to handle decision boundaries. The input data is implicitly mapped to a higher-dimensional feature space where separation becomes feasible. This transformation is performed through kernel functions without explicitly computing the coordinates in the high-dimensional space.

Common Kernel functions include:

- **Linear Kernel:** The linear kernel is suitable when the data is approximately linearly separable.

$$K(x_i, x_j) = x_i \cdot x_j$$

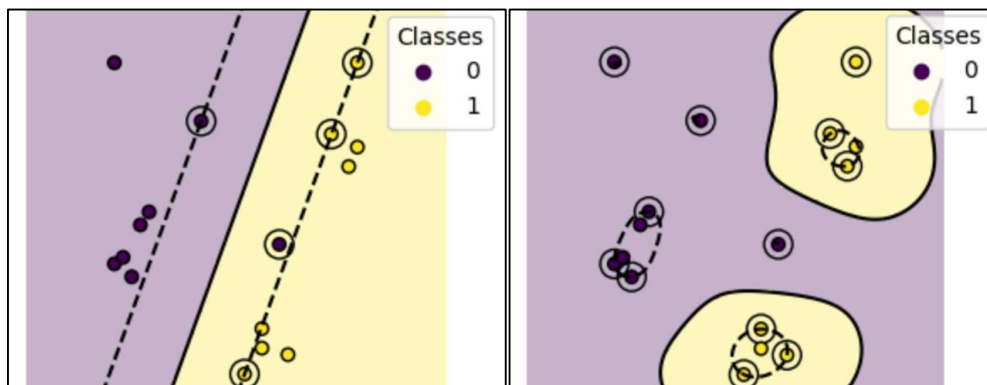
- **Radial Basis Function (RBF) Kernel:** The RBF kernel maps data into an infinite-dimensional feature space and is highly effective for capturing complex, non-linear relationships.

$$K(x_i, x_j) = e^{\frac{-|x_i - x_j|^2}{2\sigma^2}}$$

The variance (σ^2) controls the influence of individual training samples. Higher values lead to tighter decision boundaries.

The **RBF kernel** is particularly effective and is commonly used for modelling complex, non-linear relationships due to its radially localised response characteristics.

In the figure given below, Linear Kernel (left) is unable to separate a single 'Class 0' from 'Class 1' sample which RBF Kernel (right) is able to separate well. Mainly due to, RBF can get more meaningful support vectors (circled points) to classify sample points with help of better hyperplane formation.



Merits of Support Vector Machines

- **Good generalization performance:**
SVMs focus on maximizing the margin between classes, which helps the model perform well on unseen data and reduces overfitting, especially in high-dimensional datasets.
- **Works well for both linear and non-linear data:**
By using different kernel functions such as Linear and RBF, SVMs can handle simple linearly separable data as well as complex non-linear patterns effectively.

- **Uses only important data points:**
The model depends mainly on support vectors, which are the most critical data points near the decision boundary. This makes the classifier efficient and robust.

Demerits of Support Vector Machines

- **High training time for large datasets:**
SVM training can be slow and computationally expensive when the dataset is very large, particularly when non-linear kernels are used.
- **Sensitive to parameter selection:**
The performance of SVM strongly depends on choosing the right kernel and hyper-parameters. Incorrect values can lead to poor classification results.
- **Harder to interpret results:**
Unlike simpler models such as Linear Regression or Decision Trees, especially SVMs with non-linear kernels do not provide clear insights into how individual features might affect predictions.

Code Block 1: Importing Libraries

Input

```
# Numerical and data handling libraries

import numpy as np

import pandas as pd


# Visualization libraries

import matplotlib.pyplot as plt

import seaborn as sns

from mlxtend.plotting import plot_decision_regions


print("Loaded libraries")
```

Output

Loaded libraries

Code Block 2: Importing Libraries

Input

```
# Dataset utilities

from sklearn.datasets import load_wine, make_moons

# Preprocessing and model utilities

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Models

from sklearn.svm import SVC

# Evaluation metrics

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
```

```
f1_score,  
classification_report,  
confusion_matrix  
)  
  
print("Loaded Sklearn libraries")
```

Output

Loaded Sklearn Libraries

Code Block 3: Loading and Reading Data

Input

```
wine = load_wine()  
wine_df = pd.DataFrame(wine.data, columns=wine.feature_names)  
  
wine_df["target"] = wine.target  
wine_df
```

Output

index	alcohol	malic_acid	ash	alcalinity_of_ash	...
0	14.23	1.71	2.43	15.6	...
1	13.20	1.78	2.14	11.2	...
2	13.16	2.36	2.67	18.6	...
3	14.37	1.95	2.50	16.8	...
4	13.24	2.59	2.87	21.0	...
...	...	...	...	...	...

178 rows × 14 columns

Code Block 4: Loading and Reading Data

Input

```
wine_df.info()
```

Output

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 178 entries, 0 to 177  
Data columns (total 14 columns):
```

#	Column	Non-Null Count	Dtype
0	alcohol	178 non-null	float64
1	malic_acid	178 non-null	float64
2	ash	178 non-null	float64
3	alcalinity_of_ash	178 non-null	float64
4	magnesium	178 non-null	float64
5	total_phenols	178 non-null	float64
6	flavanoids	178 non-null	float64
7	nonflavanoid_phenols	178 non-null	float64
8	proanthocyanins	178 non-null	float64
9	color_intensity	178 non-null	float64
10	hue	178 non-null	float64
11	od280/od315_of_diluted_wines	178 non-null	float64
12	proline	178 non-null	float64
13	target	178 non-null	int64

```
dtypes: float64(13), int64(1)
```

```
memory usage: 19.6 KB
```

Code Block 5: Loading and Reading Data

Input

```
# Generate two-moons dataset

X_moons, y_moons = make_moons(n_samples=700,
                                noise=0.15,
                                random_state=42
                                )

moons_df = pd.DataFrame(X_moons,
                        columns=["Feature_1", "Feature_2"])

moons_df["target"] = y_moons

moons_df
```

Output

Index	Feature_1	Feature_2	target
0	-0.036777	1.017428	0
1	0.991181	-0.542893	1
2	0.135605	0.095587	1
3	0.380490	0.882966	0
4	-0.795374	0.193136	0
...	...	...	...
695	1.147087	0.520498	0
696	0.200663	0.598349	0
697	-0.384952	0.547244	0
698	0.452570	-0.174473	1
699	0.534527	0.910810	0

700 rows × 3 columns

Code Block 6: Data Analysis

Input

```
wine_df = pd.concat(
    [wine_df[["flavanoids", "color_intensity"]], wine_df["target"]],
    axis=1
)

wine_target_names = {
    0: "Barolo",
    1: "Grignolino",
    2: "Barbera"
}

wine_df["class_name"] = wine_df["target"].map(wine_target_names)
wine_df[["target", "class_name"]].value_counts()
```

Output

target	class_name	count
1	Grignolino	71
0	Barolo	59
2	Barbera	48

Name: count, dtype: int64

Code Block 7: Data Analysis

Input

```
# Visualizing Wine Dataset

plt.figure(figsize=(7, 5))

sns.scatterplot(
    data=wine_df,
    x="flavanoids",
    y="color_intensity",
    hue="class_name",
```



```

palette="Set2",

s=40,

edgecolor="black"

)

plt.title("Wine Dataset: Linearly Separable Features",

        fontsize=14,

        fontweight="bold")

plt.xlabel("Flavanoids", fontsize=12)

plt.ylabel("Color Intensity", fontsize=12)

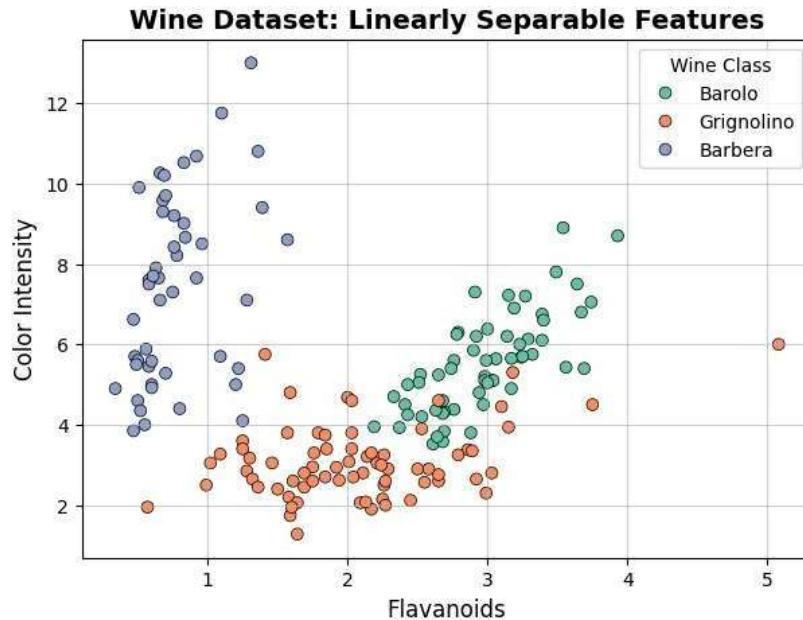
plt.legend(title="Wine Class")

plt.grid(alpha=0.6)

plt.show()

```

Output



Code Block 8: Data Analysis

Input

Histogram of Flavanoids

```
plt.figure(figsize=(10, 5))
```

```
sns.histplot(
```

```
    data=wine_df,
```

```
    x="flavanoids",
```

```
    hue="class_name",
```

```
    kde=True,
```

```
    palette="Set2",
```

```
    element="step",
```

```
    stat="density",
```

```
    common_norm=False
```

```
)
```

```
plt.title("Distribution of Flavanoids by Wine Class",
```

```
        fontsize=14,
```

```
        fontweight="bold")
```

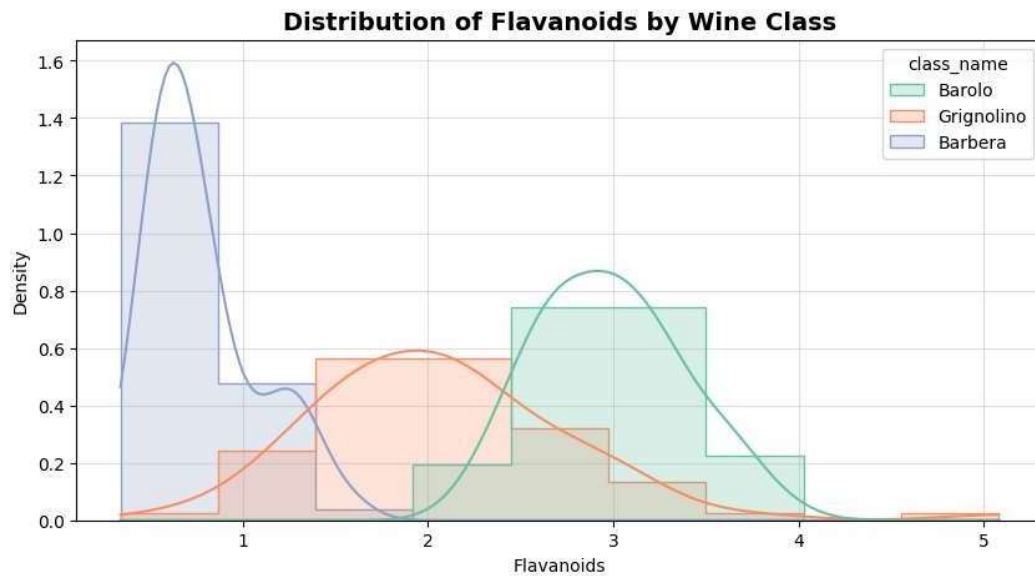
```
plt.xlabel("Flavanoids")
```

```
plt.ylabel("Density")
```

```
plt.grid(alpha=0.4)
```

```
plt.show()
```

Output



Code Block 9: Data Analysis

Input

```
# Histogram of Color Intensity
plt.figure(figsize=(10, 5))

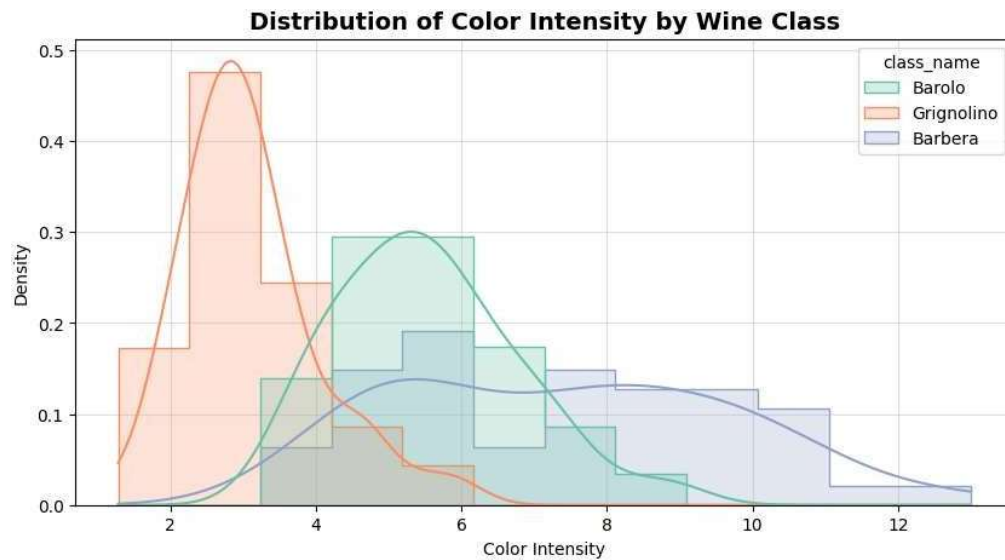
sns.histplot(
    data=wine_df,
    x="color_intensity",
    hue="class_name",
    kde=True,
    palette="Set2",
    element="step",
    stat="density",
    common_norm=False
)

plt.title("Distribution of Color Intensity by Wine Class",
          fontsize=14,
          fontweight="bold")

plt.xlabel("Color Intensity")
plt.ylabel("Density")
plt.grid(alpha=0.4)
```

```
plt.show()
```

Output



Code Block 10: Data Analysis

Input

```
# Visualizing Two Moons Dataset
```

```
plt.figure(figsize=(8,6))
```

```
sns.scatterplot(
```

```
    x=moons_df["Feature_1"],
```

```
    y=moons_df["Feature_2"],
```

```
    hue=moons_df["target"].map({0: "Blue Moon", 1: "Orange Moon"}),
```

```
    palette=["#1f77b4", "#ff7f0e"]
```

```
)
```

```
plt.title("Two Moons Dataset – Non-Linear Separability",
```

```
        fontsize=14,
```

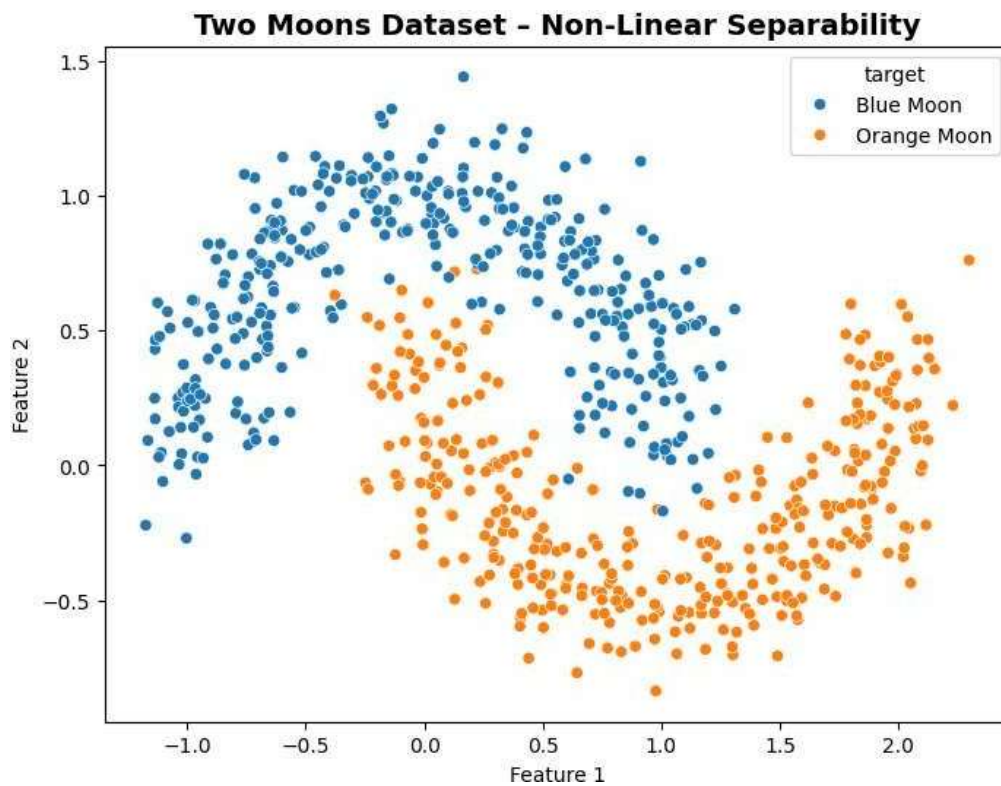
```
        fontweight='bold')
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.show()
```

Output



Code Block 11: Data Preprocessing

Input

```
X_wine = wine_df[["flavanoids", "color_intensity"]]
y_wine = wine_df["target"]

Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    X_wine, y_wine, test_size=0.2,
    random_state=42, stratify=y_wine
)

Xw_train.shape, Xw_test.shape, yw_train.shape, yw_test.shape
```

Output

```
((142, 2), (36, 2), (142,), (36,))
```

Code Block 12: Data Preprocessing

Input

```
Xm_train, Xm_test, ym_train, ym_test = train_test_split(  
    X_moons, y_moons, test_size=0.2,  
    random_state=42, stratify=y_moons  
)  
  
Xm_train.shape, Xm_test.shape, ym_train.shape, ym_test.shape
```

Output

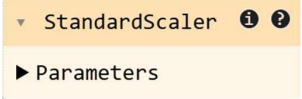
```
((560, 2), (140, 2), (560,), (140,))
```

Code Block 13: Data Preprocessing

Input

```
# Feature Scaling  
  
scaler = StandardScaler()  
  
scaler
```

Output



The output shows a Jupyter Notebook cell with the text 'StandardScaler' followed by an information icon (i) and a help icon (?). Below this, there is a section titled 'Parameters' with a right-pointing triangle icon.

Code Block 14: Data Preprocessing

Input

```
Xw_train_scaled = scaler.fit_transform(Xw_train)  
  
Xw_test_scaled = scaler.transform(Xw_test)  
  
print("Wine data scaled successfully!")
```

Output

Wine data scaled successfully!

Code Block 15: Data Preprocessing

Input

Xw_train[:5]

Output

Index	flavanoids	color_intensity
-----	-----	-----
36	2.68	4.60
30	3.25	5.70
26	2.94	4.80
12	2.76	5.60
148	0.76	8.42

Code Block 16: Data Preprocessing

Input

Xw_train_scaled[:5]

Output

```
array([[ 0.73229212, -0.16746725],
       [ 1.33318146,  0.30530313],
       [ 1.006382  , -0.081509  ],
       [ 0.81662747,  0.262324  ],
       [-1.29175618,  1.47433535]])
```

Code Block 17: Data Preprocessing

Input

```
Xm_train_scaled = scaler.fit_transform(Xm_train)
Xm_test_scaled = scaler.transform(Xm_test)
print("Two Moons data scaled successfully!")
```

Output

Two Moons data scaled successfully!

Code Block 18: Data Preprocessing

Input

```
Xm_train[:5]
```

Output

```
array([[ 1.3723603, -0.55081882],
       [ 0.63613117,  0.78345978],
       [-0.43531115,  0.96004336],
       [ 0.41718644,  1.17721089],
       [-1.13531808,  0.17220389]])
```

Code Block 19: Data Preprocessing

Input

```
Xm_train_scaled[:5]
```

Output

```
array([[ 1.01466867, -1.57697415],
       [ 0.17320777,  1.01862443],
       [-1.05137949,  1.36213595],
       [-0.07703149,  1.78459625],
       [-1.85144083, -0.17046376]])
```

Code Block 20: Model Training

Input

```
svm_linear_wine = SVC(kernel="linear")  
  
svm_linear_wine.fit(Xw_train_scaled, yw_train)  
  
svm_linear_wine
```

Output



Code Block 21: Model Training

Input

```
# Training SVMs with linear and RBF kernels on  
# two-moons toy dataset
```

```
svm_linear_moon = SVC(kernel="linear")  
svm_rbf_moon = SVC(kernel="rbf")  
  
svm_linear_moon.fit(Xm_train_scaled, ym_train)  
svm_rbf_moon.fit(Xm_train_scaled, ym_train)  
print("Model trained")
```

Output

Model trained

Code Block 22: Model Evaluation

Input

```
# Wine dataset case

wine_preds = svm_linear_wine.predict(Xw_test_scaled)

wine_preds
```

Output

```
array([0, 2, 0, 0, 1, 0, 0, 0, 1, 2, 1, 2, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1,
       0, 0, 0, 1, 0, 2, 1, 2, 0, 2, 1, 2, 2, 2])
```

Code Block 23: Model Evaluation

Input

```
print("Accuracy:", accuracy_score(yw_test, wine_preds))

print("Precision:", precision_score(yw_test, wine_preds,
                                   average="macro"))

print("Recall:", recall_score(yw_test, wine_preds,
                              average="macro"))

print("F1 Score:", f1_score(yw_test, wine_preds,
                            average="macro"))

print("\nClassification Report:\n")

print(classification_report(yw_test, wine_preds))
```

Output

Accuracy: 0.8611
Precision: 0.8773
Recall: 0.8675
F1 Score: 0.8694

Classification Report:

	precision	recall	f1-score	support
	-----	-----	-----	-----
0	0.79	0.92	0.85	12
1	0.85	0.79	0.81	14
2	1.00	0.90	0.95	10
accuracy			0.86	36
macro avg	0.88	0.87	0.87	36
weighted avg	0.87	0.86	0.86	36

Code Block 24: Model Evaluation

Input

```
# Confusion Matrix

labels = list(wine_target_names.keys())

class_names = list(wine_target_names.values())

plt.figure(figsize=(6, 5))

sns.heatmap(

    confusion_matrix(yw_test, wine_preds, labels=labels),

    annot=True,

    fmt="d",

    cmap="Blues",

    xticklabels=class_names,

    yticklabels=class_names

)

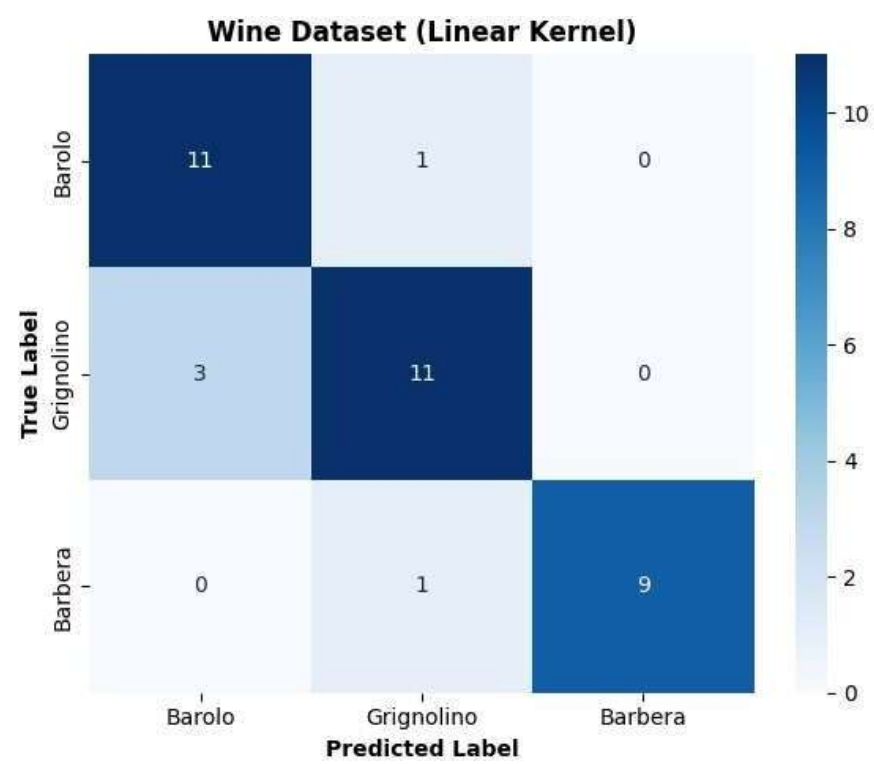
plt.title("Wine Dataset (Linear Kernel)", fontweight='bold')

plt.xlabel("Predicted Label", fontweight='bold')

plt.ylabel("True Label", fontweight='bold')
```

```
plt.tight_layout()
plt.show()
```

Output



Code Block 25: Model Evaluation

Input

```
# Two moon dataset case
moon_linear_preds = svm_linear_moon.predict(Xm_test_scaled)
moon_linear_preds
```

Output

```
array([0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0,
      0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1,
      1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1,
      0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0,
      1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0,
      1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0,
      0, 0, 1, 0, 0, 1, 1, 1])
```

Code Block 26: Model Evaluation

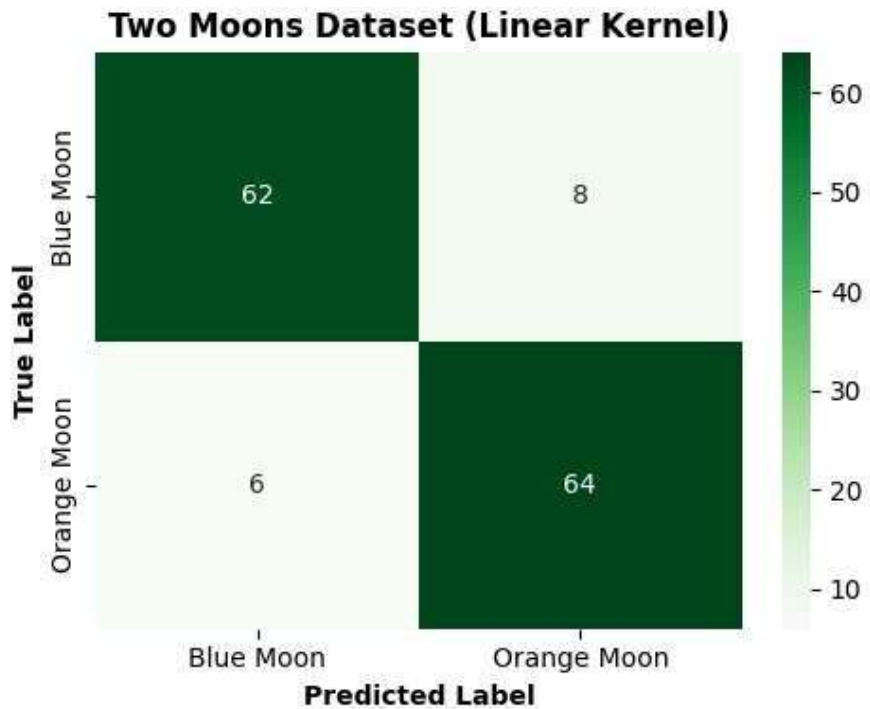
Input

```
# Confusion Matrix
class_names = ['Blue Moon', 'Orange Moon']

plt.figure(figsize=(5, 4))
sns.heatmap(
    confusion_matrix(ym_test, moon_linear_preds),
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title("Two Moons Dataset (Linear Kernel)", fontweight='bold')
plt.xlabel("Predicted Label", fontweight='bold')
plt.ylabel("True Label", fontweight='bold')
plt.tight_layout()
plt.show()
```

Output



Code Block 27: Model Evaluation

Input

```
print("Accuracy:", accuracy_score(ym_test, moon_linear_preds))

print("Precision:", precision_score(ym_test,
    moon_linear_preds,
    average="macro"))

print("Recall:", recall_score(ym_test,
    moon_linear_preds,
    average="macro"))

print("F1 Score:", f1_score(ym_test,
    moon_linear_preds,
    average="macro"))

print("\nClassification Report:\n")

print(classification_report(ym_test, moon_linear_preds))
```

Output
Accuracy: 0.9000
Precision: 0.9003
Recall: 0.9000
F1 Score: 0.9000

Classification Report:

	precision	recall	f1-score	support
	-----	-----	-----	-----
0	0.91	0.89	0.90	70
1	0.89	0.91	0.90	70
accuracy			0.90	140
macro avg	0.90	0.90	0.90	140
weighted avg	0.90	0.90	0.90	140

Code Block 28: Model Evaluation

Input

```
# Two moon dataset case  
  
moon_rbf_preds = svm_rbf_moon.predict(Xm_test_scaled)  
  
moon_rbf_preds
```

Output

```
array([0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1,  
       1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1,  
       0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0,  
       1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0,  
       1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0,  
       1, 1, 1, 0, 1, 1, 0, 1])
```

Code Block 29: Model Evaluation

Input

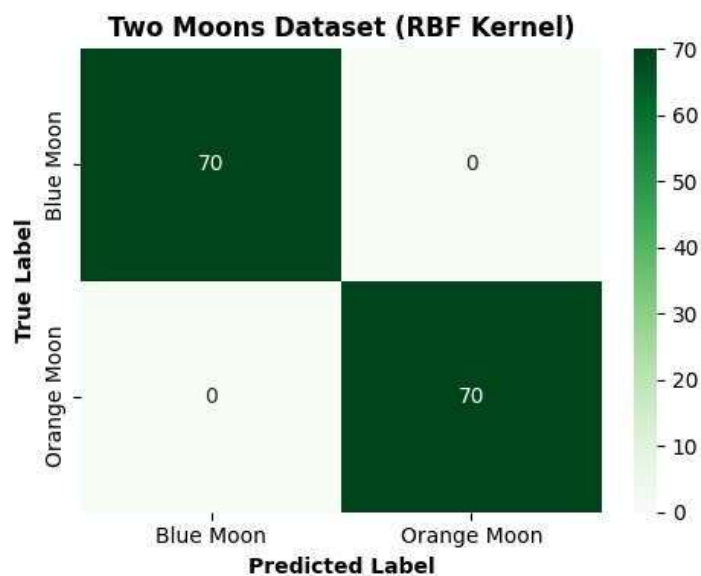
```
class_names = ['Blue Moon', 'Orange Moon']

plt.figure(figsize=(5, 4))

sns.heatmap(
    confusion_matrix(ym_test, moon_rbf_preds),
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title("Two Moons Dataset (RBF Kernel)", fontweight='bold')
plt.xlabel("Predicted Label", fontweight='bold')
plt.ylabel("True Label", fontweight='bold')
plt.tight_layout()
plt.show()
```

Output



Code Block 30: Model Evaluation

Input

```
print("Accuracy:", accuracy_score(ym_test, moon_rbf_preds))
print("Precision:", precision_score(ym_test,
    moon_rbf_preds,
    average="macro"))
print("Recall:", recall_score(ym_test,
    moon_rbf_preds,
    average="macro"))
print("F1 Score:", f1_score(ym_test,
    moon_rbf_preds,
    average="macro"))

print("\nClassification Report:\n")
print(classification_report(ym_test, moon_rbf_preds))
```

Output

```
Accuracy:  1.0000
Precision:  1.0000
Recall:    1.0000
F1 Score:   1.0000
```

Classification Report:

	precision	recall	f1-score	support
	-----	-----	-----	-----
0	1.00	1.00	1.00	70
1	1.00	1.00	1.00	70
accuracy			1.00	140
macro avg	1.00	1.00	1.00	140
weighted avg	1.00	1.00	1.00	140

Code Block 31: Model Evaluation

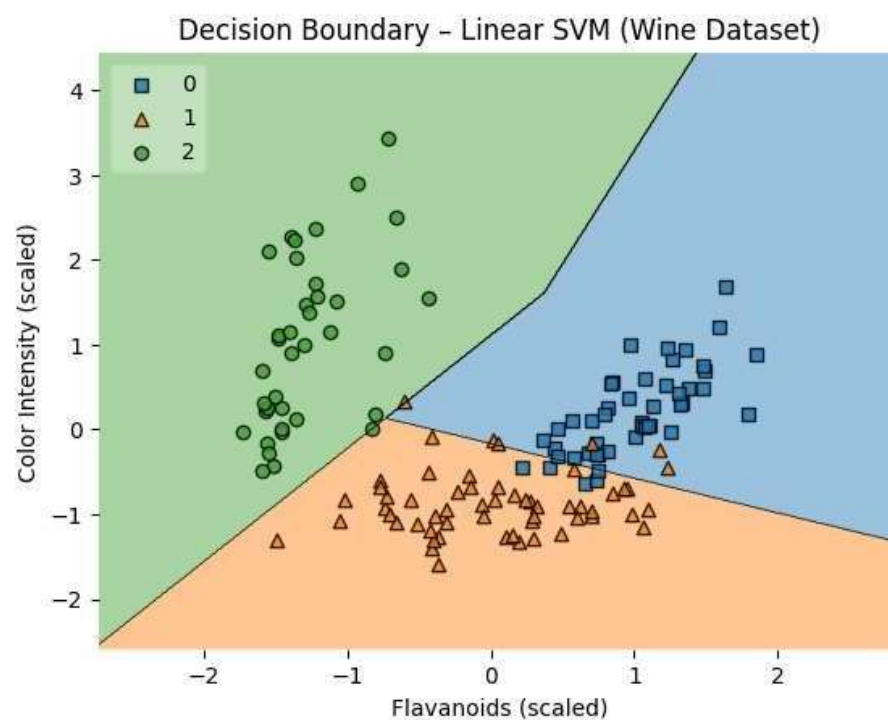
Input

```
# Decision Boundaries – Wine

plot_decision_regions(
    Xw_train_scaled,
    yw_train.values,
    clf=svm_linear_wine,
    legend=2
)

plt.title("Decision Boundary – Linear SVM (Wine Dataset)")
plt.xlabel("Flavanoids (scaled)")
plt.ylabel("Color Intensity (scaled)")
plt.show()
```

Output



Code Block 32: Model Evaluation

Input

Decision Boundaries – Two Moons

```
plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)
```

```
plot_decision_regions(Xm_train_scaled, ym_train,  
                     clf=svm_linear_moon)
```

```
plt.title("Linear Kernel – Blue vs Orange Moon")
```

```
plt.subplot(1,2,2)
```

```
plot_decision_regions(Xm_train_scaled, ym_train,  
                     clf=svm_rbf_moon)
```

```
plt.title("RBF Kernel – Blue vs Orange Moon")
```

```
plt.show()
```

Output

